YZV 322E - Applied Data Engineering

Containerized ETL Pipeline for Restaurant Trend Analysis

Şeyma Betül İSKENDER
150220342

Ahmet Selim ÖZEL
150230717

Team : DockerMaster

Project Overview

What We Built?

A fully containerized, end-to-end ETL pipeline that ingests multi-source restaurant data, applies layered transformations, and delivers interactive dashboards.

Entire system starts with one command:
docker compose up --build

Research Questions

- Which cuisines score highest? Does price predict ratings?
- Do restaurants with online delivery get better ratings?
- Where are cheap but highly-rated restaurants concentrated?

Tools & Stack

PostgreSQL
Relational Staging

pgAdmin
DB Administration

Apache Airflow
Orchestration (DAG)

Elasticsearch
Indexing

Kibana
Dashboard

Project Goals & Problem

Goal: transform raw, multi-source restaurant data into queryable, map-based analytical outputs-reproducibly, on a student laptop.

DATA SOURCES

1. Zomato Datasets 9,551 real restaurant records including cuisine types, ratings, average cost, and location coordinates.

2. Synthetic Orders Live order simulation generated with Python Faker — timestamped events for hourly demand analysis on Kibana.

Why Do We Need This Pipeline?

- **Heterogeneous Sources — Zomato Global and Zomato Bangalore arrive with different schemas and field names. Manual unification is error-prone and not reproducible at scale.**
- **No Unified Search — Traditional relational databases cannot efficiently filter and search across large restaurant datasets. An Elasticsearch index is required.**
- **No Automated Refresh — Ad-hoc scripts run once and break silently. A scheduled, idempotent Airflow DAG ensures summaries stay consistent across every re-run. (Apache Airflow)**

End-to-End ETL Architecture

Data Sources

Zomato CSV
Faker Producer

▶

PostgreSQL +pgAdmin

Staging & Admin

▶

Apache Airflow

Orchestrate & Aggregate

▶

Elasticsearch +Kibana

Index & Visualize

Single Command Start

docker compose up --build

No Host Dependencies

Python, Java — all containerized

Secrets via .env

Zero hardcoded credentials

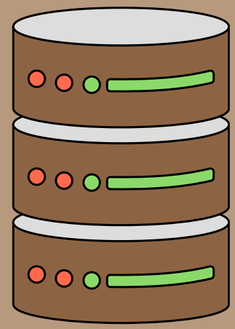
Named Docker Volumes

Data persists across restarts

All services communicate via pipeline-net Docker bridge network — no service depends on localhost or the host IP.

PostgreSQL & pgAdmin — Stage & Administer

What is PostgreSQL used for?



It's our main data storage layer. All cleaned restaurant data lands here first, then gets summarized automatically every day.

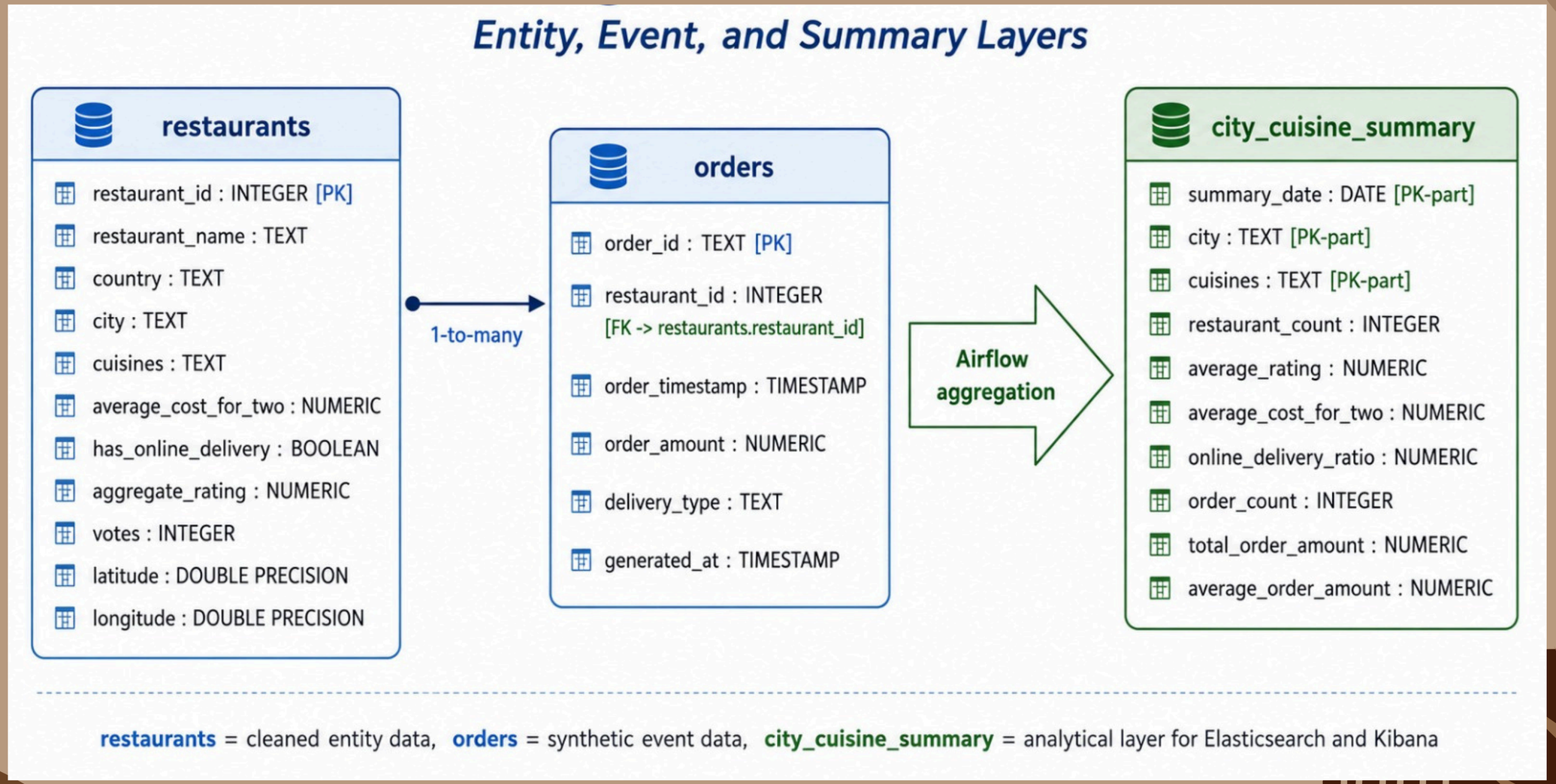
raw_restaurants - The main data table. Every validated record is stored here with a timestamp showing exactly when it arrived.

city_summary - A daily summary table created automatically by Airflow. Groups restaurants by city and cuisine — calculates average ratings, costs, and delivery rates. Running it twice on the same day gives the same result, no duplicates.

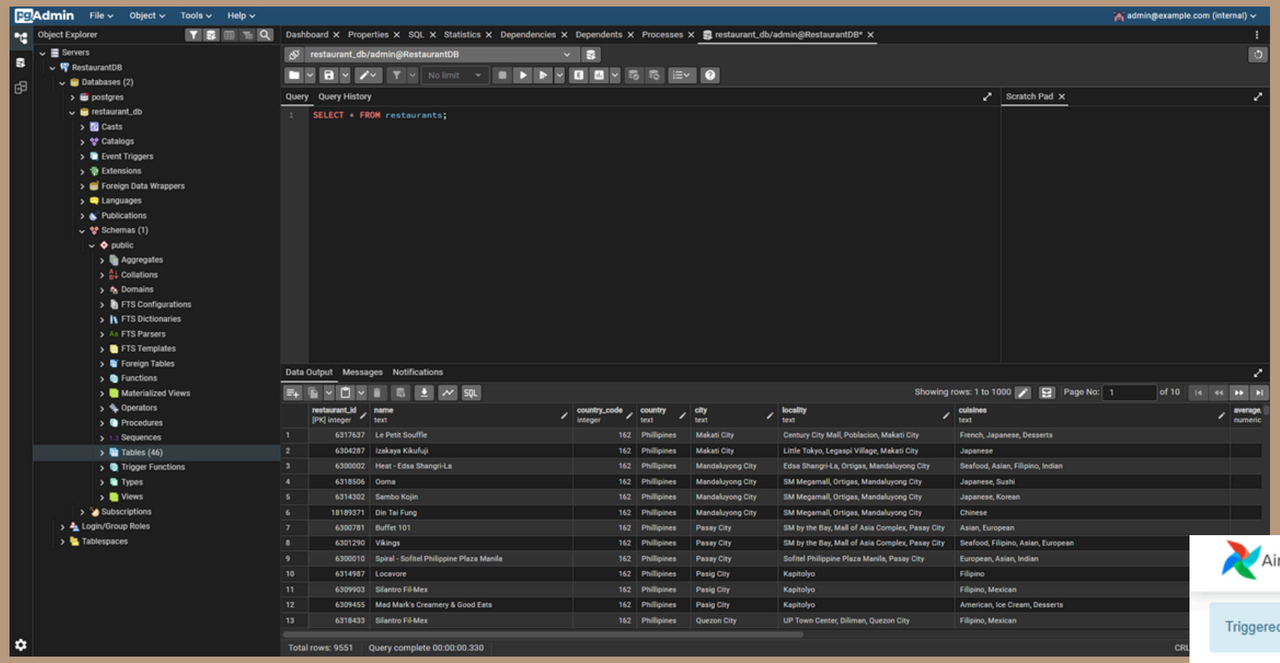
pgAdmin - A web-based database interface that runs inside Docker. No extra software needed — open a browser and query the database directly.

Think of it this way: raw_restaurants is the raw storage, city_summary is the clean report, and pgAdmin is the control panel.

PostgreSQL Schema

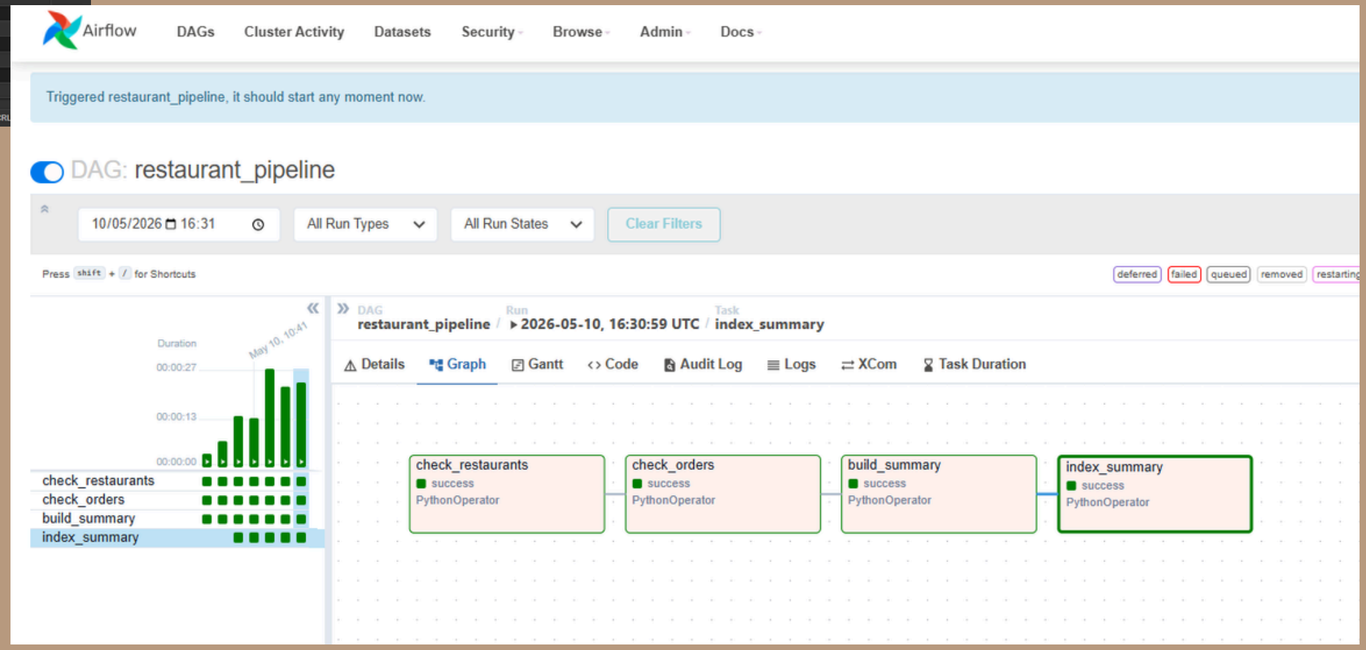


Demo Screenshots



pgAdmin — 9,551 rows in restaurants table

Airflow — all 4 DAG tasks completed successfully



Apache Airflow — Orchestrate the Batch Layer

DAG Parameters	
Schedule	@daily (00:05)
Start Date	2026-05-01
Catchup	False
Retry	2× · 5-min delay
Timeout	30 minutes

What does Apache Airflow do?

It's our automated scheduler. Every night at 00:05, it runs a 4-step job automatically — no manual intervention needed.

Step1 - Check for new data Looks at the database and asks: "Did any new restaurant records arrive since yesterday?"

Step 2 - Summarize by city Groups all restaurants by city and cuisine type. Calculates average rating, average cost, and delivery rate for each group.

Step 3 - Send to Elasticsearch Pushes the cleaned and summarized data to Elasticsearch so it appears on the Kibana map dashboard.

Step 4 - Log the results Records how many rows were processed that night — for tracking and auditing.

If a step fails, Airflow automatically retries twice with a 5-minute wait. The whole job times out after 30 minutes if something goes wrong.

Elasticsearch & Kibana — Index, Search & Visualize

Elasticsearch — How We Store the Data

Unlike regular databases, Elasticsearch stores location coordinates as geo_point — enabling map-based queries like "show me restaurants within 5 km."

📍 Location
Coordinates stored as geo_point for map queries

🍽️ Name / Cuisine
Searchable by full text or exact match

⭐ Rating
Filter and sort restaurants by score

💰 Avg Cost
Query by price range

🛵 Online Delivery
Filter delivery vs dine-in

🏙️ City
Group results by city

Kibana — 6 Dashboard Panels

🗺️ Geographic Heat Map
Restaurants on a live map, colored by rating

📊 Cuisine Rating Bar Chart
Top 15 cuisines ranked by average customer score

⚡ Price vs Rating Scatter
Does spending more mean better food?

🥧 Online Delivery Pie
What % of restaurants offer delivery?

📋 City Summary Table
City-level stats: count, avg rating, avg cost

📈 Hourly Order Volume
When are restaurants busiest? (synthetic stream)

The System Fails Gracefully — By Design

Airflow

Risk: Task failure mid-pipeline

→ Automatically retries twice with a 5-minute wait before giving up.

PostgreSQL

Risk: Container restart loses data

→ Data is stored on a separate volume — even if Docker shuts down, everything is safe.

Elasticsearch

Risk: Wrong coordinate format

→ Coordinates are validated before indexing — bad data never reaches Elasticsearch.

Airflow → Elasticsearch

Risk: Duplicate indexing on re-run

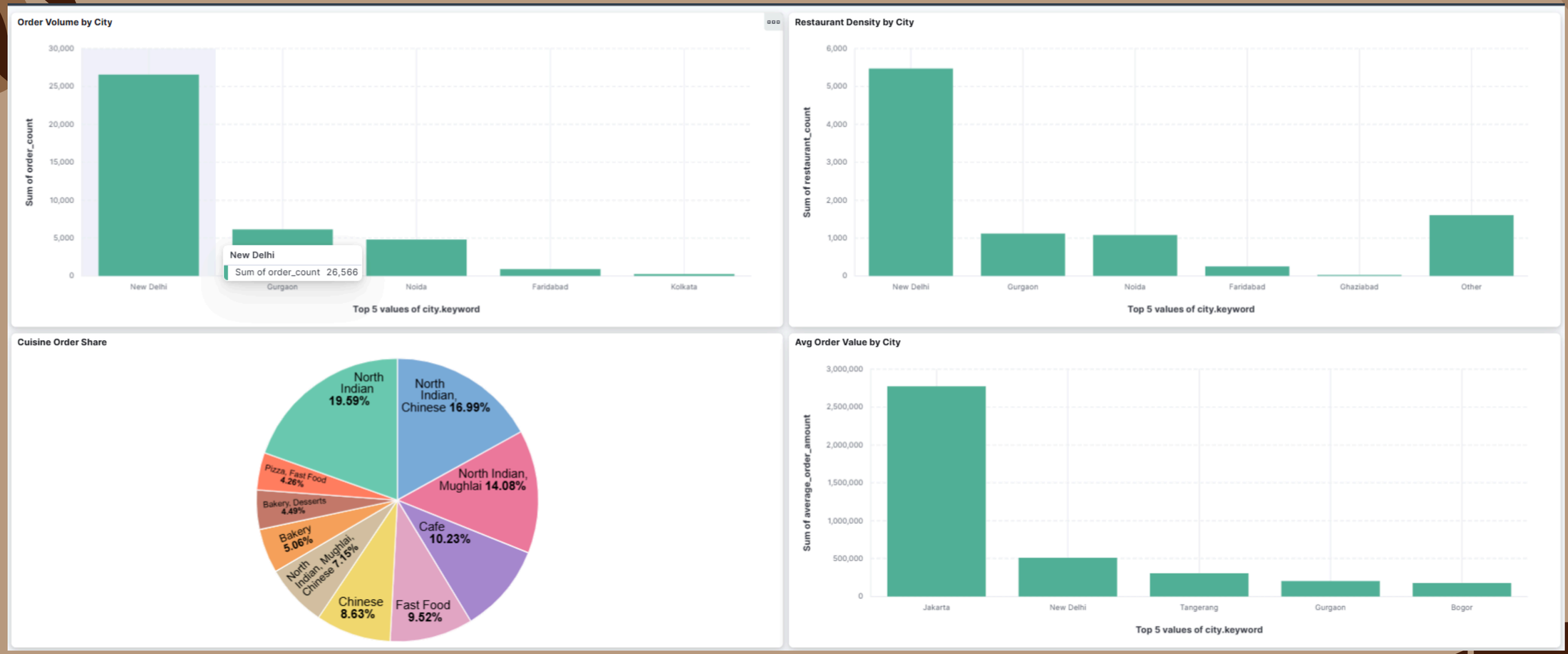
→ If the same data is sent twice, it simply overwrites — no duplicate records created.

Docker Network

Risk: localhost / host-IP dependency

→ All services communicate through an internal network — no dependency on the host machine.

Results & Dashboard



İNew Delhi dominates in both order volume and restaurant count. North Indian cuisine leads with 19.59% of all orders. Jakarta's high average order value is due to currency differences in the synthetic dataset.

FUTURE WORK

- Replace batch processing with real-time streaming using Apache Kafka.
- Expand the dataset to cover global markets beyond India.
- Train a ML model to predict restaurant ratings.
- Add automated alerts for pipeline failures and data quality issues.
- Build a simple web interface for non-technical users to explore the data.

THANK YOU